

CSA0613-Design and Analysis of Algorithms

NAME: Safiya B

REG NO: 192411043

Experiment 33: Binary Search Performance Under Different Hardware Architectures

Aim

To analyze the performance of the Binary Search algorithm under different processor speeds by measuring execution time and comparing the results.

Algorithm

1. Create a sorted array.
2. Read the key element to search.
3. Record the start time.
4. Apply Binary Search.
5. Record the end time.
6. Calculate execution time.
7. Repeat for different simulated processor speeds.
8. Compare the execution times and analyze the results.

Source Code

```
1 import time
2 def binary_search(a, key):
3     low, high = 0, len(a) - 1
4     while low <= high:
5         mid = (low + high) // 2
6         if a[mid] == key:
7             return mid
8         elif a[mid] < key:
9             low = mid + 1
10        else:
11            high = mid - 1
12    return -1
13 a = list(range(1, 100001))
14 key = 98765
15 speeds = [1, 2, 4, 8]
16 print("Binary Search Performance")
17 print("-----")
18 for speed in speeds:
19     start = time.perf_counter()
20     for _ in range(speed * 1000):
21         binary_search(a, key)
22     end = time.perf_counter()
23     time_taken = (end - start) / (speed * 1000)
24
25     print("Processor Speed:", speed, "GHz")
26     print("Execution Time: {:.8f} seconds".format(time_taken
27         ))
28     print()
```

Sample Output

```
Binary Search Performance
-----
Processor Speed: 1 GHz
Execution Time: 0.00000217 seconds

Processor Speed: 2 GHz
Execution Time: 0.00000216 seconds

Processor Speed: 4 GHz
Execution Time: 0.00000226 seconds

Processor Speed: 8 GHz
Execution Time: 0.00000301 seconds

=== Code Execution Successful ===
```

Performance Comparison

Processor	Clock Speed	Execution Time
Processor A	2.0 GHz	0.000005 s
F Processor B	3.0 GHz	0.000003 s
Processor C	4.0 GHz	0.000002 s

(Values are sample results. Record your actual execution times if you test on different systems.)

Analysis

- Binary Search requires the data to be sorted.
- Faster processors reduce execution time.
- The number of comparisons remains nearly the same on all systems.
- Hardware improves speed but does not change the algorithm's complexity.

- Binary Search performs efficiently even on slower processors.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$

Space Complexity: $O(1)$

Conclusion

Binary Search is a highly efficient searching algorithm. Although execution time decreases on faster processors, its logarithmic time complexity remains unchanged. This demonstrates that Binary Search is robust and performs efficiently across different hardware architectures.